COURSE WEB STUDY

بناء API واضح بـ Node وNest

Request → Validation → Service → Response

BACKEND WITH NODE & NEST

شرح عربي · أمثلة إنجليزية · تمارين عملية

فهرس الكتاب

1 العقد قبل الكود 3
2 افصل المسؤوليات في Nest 3
3 اربط الخدمات الخارجية بأمان 4
4 مراجعة قبل التسليم 4
5 رحلة الطلب: من HTTP إلى استجابة مفهومة 6
6 validation والأسرار والحدود الآمنة 7
7 مشروع API: جلسات مذاكرة قابلة للاختبار 8

كيف تستخدم هذا الدليل؟

الدليل يركز على الحدود الواضحة: ما يدخل إلى التطبيق، أين تتحقق منه، وأي نتيجة تعود إلى الواجهة. اكتب عقد endpoint قبل التفاصيل، ولا تسمح لأي سر أن يصل إلى المتصفح.

نتائج التعلّم

- تصميم عقد طلب واستجابة صغير
- فصل Controller عن Service
- تقديم أخطاء آمنة ومفهومة

1 العقد قبل الكود

قبل أن تكتب endpoint، دوّن المسار والطريقة وشكل البيانات. العقد لا يعني وثيقة ضخمة؛ يكفي أن تعرف الحقول المطلوبة، الرد الناجح، ورد التحقق الفاشل. الواجهة الأمامية لا يجب أن تخمّن أسماء الحقول أو معنى الخطأ.

```
GET /api/study-domains/backend/challenges?difficulty=beginner

200 OK
{ "domain": "backend", "items": [{ "title": "Build a health endpoint" }] }
```

INPUT
طلب محدود الحقول ومتحقق منه عند أول نقطة دخول.

OUTPUT
رد ثابت الشكل ورسالة خطأ لا تكشف تفاصيل داخلية.

تمرين: عقد ملاحظة

صمّم `POST /notes`: ما الحقول الإلزامية؟ ما الاستجابة بعد الإنشاء؟ وما رسالة التحقق عند غياب العنوان؟

2 افصل المسؤوليات في Nest

وظيفة Controller هي استقبال الطلب وتمريره ثم إعادة النتيجة. قواعد العمل تعيش في Service. أما DTO فهو الباب الذي يحدد شكل المدخلات. هذا الفصل يجعل كل جزء أقصر وأسهل في الاختبار من endpoint يحمل كل شيء.

```
@Get(':slug')
findOne(@Param('slug') slug: string) {
```

```
  return this.catalogService.findDomain(slug);
}

@Injectable()
export class CatalogService {
  findDomain(slug: string) {
    return this.domains.find((item) => item.slug === slug);
  }
}
```

لا تخلط `res.status()` والمنطق ومعالجة مصادر البيانات في نفس الدالة إلا عند الضرورة. إذا كان اسم الدالة لا يشرح ما تفعله، غالباً تحتاج إلى فصل أصغر.

تمرين: ثلاث حالات

اكتب اختباراً لمسار التحديات يتعامل مع مستوى صحيح، مستوى غير معروف، ومجال غير موجود. حدّد status code المناسب لكل حالة قبل كتابة assertion.

3 اربط الخدمات الخارجية بأمان

مفتاح AI أو قاعدة بيانات ليس إعداداً للعميل. احتفظ به في متغيرات بيئة الخادم، ومرر إلى الواجهة نتيجة مختصرة فقط. إذا لم تكن الخدمة مهيأة، أعد رسالة واضحة يمكن للمستخدم فهمها، ولا تحاول اختراع إجابة بديلة.

```
if (!process.env.AI_API_KEY) {
  throw new ServiceUnavailableException(
    'AI tutor is not configured on this server yet.',
  );
}
```

أضف حدوداً للطلبات عندما تكون الخدمة مكلفة أو يمكن إساءة استخدامها، ثم اختبر مسار النجاح ومسار غياب الإعداد. الأمان ليس ميزة تضيفها في النهاية؛ هو قرار في مكان اتصال البيانات أولاً.

تمرين: فشل مفهوم

اختبر أن endpoint المحادثة يرجع 503 عندما يكون المفتاح غير موجود. تأكد أن الرد لا يعرض اسم المفتاح ولا stack trace.

4 مراجعة قبل التسليم

- هل العقد موثق بمثال واحد على الأقل؟

- هل التحقق يحدث قبل قواعد العمل؟
- هل الأسرار على الخادم فقط؟
- هل يمكن للواجهة عرض رسالة الخطأ كما هي؟

5 رحلة الطلب: من HTTP إلى استجابة مفهومة

كل request يصل بعنوان ومسار وطريقة وبيانات. لا تقفز مباشرة إلى مصدر البيانات: تحقق من المدخل، نفّذ منطق المجال، ثم أعد status مناسباً ورسالة لا تكشف تفاصيل داخلية. استخدم 400 لمدخل خاطئ و404 لمورد غير موجود؛ لا تعد 200 فقط لأن الكود لم ينهَر.

```
@Post()
create(@Body() dto: CreateNoteDto) {
  return this.notesService.create(dto);
}
```

افصل Controller عن Service. الـ Controller يترجم HTTP، والـ Service يحمل منطق العمل. هذه الحدود تجعل اختبار الجزء المهم ممكناً دون تشغيل خادم كامل كل مرة.

تمرين: استجابة صادقة

أنشئ endpoint لحفظ ملاحظة. جرّب نصاً فارغاً، ثم ملاحظة صحيحة، ثم id غير موجود. دوّن status والرسالة المتوقعة لكل حالة.

6 validation والأسرار والحدود الآمنة

التحقق ليس خطوة تجميلية. البيانات القادمة من أي عميل غير موثوقة حتى لو كان العميل واجهتك أنت. حدد الشكل المتوقع، ارفض الزائد أو غير الصالح، وسجّل أخطاء مفيدة للمطور من دون إرجاع stack trace للمستخدم.

المفتاح السري لا يوضع في NEXT_PUBLIC_ ولا في Git. احتفظ به في متغير بيئة للخادم، وقدّم للواجهة فقط ما تحتاجه من نتيجة آمنة. عندما يتطلب مزود خارجي مفتاحاً، اجعل Nest هو الوسيط بدلاً من كشف السر في المتصفح.

تمرين: خريطة أسرار

اكتب قائمة بثلاث قيم يجب أن تبقى على الخادم: مفتاح AI وعنوان قاعدة بيانات وسر توقيع. اشرح في سطر واحد ماذا يرى المتصفح بدلاً من كل قيمة.

7 مشروع API: جلسات مذاكرة قابلة للاختبار

ابنِ مورد `study-sessions` بعمليات create وlist. ابدأ بمخزن مؤقت داخل Service، ثم استبدله بقاعدة بيانات عندما تصبح الحدود واختباراتك مستقرة. اكتب الحالات التي ستختبرها قبل التنفيذ: مدة سالبة، عنوان فارغ، مورد غير موجود، وقائمة فارغة.

ROUTE

اختر اسماً يجمع مورداً لا فعلاً: `./study-sessions`

DTO

صف المدخل المتوقع، ثم ارفض البيانات غير المتوافقة مبكراً.

SERVICE

ضع الحسابات والتخزين المؤقت في مكان واحد قابل للاختبار.

TEST

اختبر النجاح والفشل عبر HTTP قبل ربط أي واجهة.